

Topic 8.4

8.4: Exception Hierarchies and Passing Data

We can create hierarchies of exceptions. In this case, our *catch* clauses should be ordered from *most specific* to *most general*. We can also transmit data from the point where the exception occurred to the point where it is handled by including appropriate members in our exceptions.

The following program illustrates exceptions hierarchies and data passed within exceptions.

```
// MODULE:      dynarr3d.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        96/04/11
//
// PURPOSE:
//   Driver for dynarr3 exercise.
//   Illustrate exceptions hierarchies and
//   data passed within exceptions.

#include      <iostream.h>
#include      <stdlib.h>
#include      <string.h>

class DynamicIntArray
{
public:
    DynamicIntArray( size_t size,
                    const char * const name );
    ~DynamicIntArray(void);
    int& operator[]( long subscript );
    size_t size(void);

class SubscriptError // Exception class.
{
public:
    SubscriptError (char * array, long subscr )
    { strcpy(array_name,array);
      bad_subscript = subscr; };
    const char * ArrayName()
    { return array_name; };
    long BadSubscript() {return bad_subscript;};

private:
    char array_name[80];
    long bad_subscript;
};

class SubscriptTooSmall : public SubscriptError
{
public:
    SubscriptTooSmall (char * array,
                      long subscr ) :
        SubscriptError(array,subscr) {};
};

class SubscriptTooLarge : public SubscriptError
{
public:
```

```

        SubscriptTooLarge (char * array,
            long subscr ) :
        SubscriptError(array,subscr) {};
};

private:
size_t array_size;    // Size of the
                    // dynamic array.
char   array_name[80]; // For educational
                    // purposes only.
int    *data;         // Pointer to dynamic
                    // memory for the array.

};

// Constructor.

DynamicIntArray::DynamicIntArray
( size_t size, const char * const name )
{
    long i; // Loop variable.

    cout << "constructor called for " << name
        << endl;
    data = new int[size];
    if (! data)
    {
        cerr << "*** Memory allocation failed.\n";
        exit(1);
    }
    array_size = size;
    for (i=0; i<array_size; ++i)
        data[i] = 0;
    strcpy(array_name,name);
}

// Destructor.

DynamicIntArray::~DynamicIntArray(void)
{
    delete data;
    array_size = 0;
    cout << "Destructor called for " << array_name
        << endl;
}

// Accessor function which returns a reference
// to an element of the dynamically-allocated
// memory.

int& DynamicIntArray::operator[](long subscript )
{
    if (subscript < 0)
        throw SubscriptTooSmall
            (this->array_name,subscript);
    else if (subscript > (array_size - 1))
        throw SubscriptTooLarge
            (this->array_name,subscript);
    return data[subscript];
}

// Inquire the size of the array.

size_t DynamicIntArray::size(void)
{
    return array_size;
}

```

```

void main(void)
{
    DynamicIntArray x(23,"x");
    long            i;
    long            out_of_range;

    for (i=0; i<23; ++i)
        x[i] = i * i;

    for (i=1; i<=2; ++i)
    {
        if (i==1)
            out_of_range = 27;
        else
            out_of_range = -3;
        try
        {
            cout << "x[13]=" << x[13] << endl;
            cout << x[out_of_range] << endl;
        }
        catch (DynamicIntArray::SubscriptTooLarge&
                theException)
        {
            cout << "*** Error accessing array "
                  << theException.array_name << endl;
            MODIFICATION: ArrayName()
            cout << "*** Subscript "
                  << theException.bad_subscript <<
                  " is too large.\n";
        }
        catch(DynamicIntArray::SubscriptError&
                theException)
        {
            cout << "*** Error accessing array "
                  << theException.array_name << endl;
            cout << "*** Subscript "
                  << theException.bad_subscript <<
                  " out of range.\n";
        }
    }
}

```

Output from dynarr3d.cpp:

```

constructor called for x
x[13]=169
*** Error accessing array x
*** Subscript 27 is too large.
x[13]=169
*** Error accessing array x
*** Subscript -3 out of range.
Destructor called for x

```

Topic 8.5 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-8-dot-5>)